

CSCI3100 Software Engineering

Design & Implementation Documentation v1.2

Author(s):

Archie Hamilton - [1155256512]

Chun Wang YIP - [1155192968]

Luke Carroll - [1155255972]

Member(s):

Hou Fong Chan - [1155200045]

Faculty of Engineering

Department of Computer Science and Engineering

The Chinese University of Hong Kong



TaskFlow



1. Introduction

1.1 Project Overview

TaskFlow is a web-based productivity system for knowledge workers. It is inspired by Jira's task tracking but intentionally focuses on:

- **Minimalist UI/UX:** fast onboarding; low cognitive load.
- **Team-scoped roles + gated access:** an admin key mechanism determines which team a user can administer.
- **Personal Canvas** (differentiator): a private brainstorming board for diagrams/notes (e.g., UML/ER sketches) integrated directly into the product.

TaskFlow supports:

- Authentication: email OTP login/signup; optional GitHub OAuth login; logout
- Licence/team gating: activation key creates/authorizes teams and admin features
- Team administration: owner/admin manage membership; owner deletes team
- Tasks: create/edit/assign/status changes; personal and shared task rules
- Views: calendar view, list view, completion view; saved view preferences
- Dashboard: activity feed + tasks due today; quick status updates
- Files: folder tree + list/grid; upload/preview/download; visibility controls
- Canvas: personal board with nodes/connectors; undo/redo; autosave; export

1.2 Scope

TaskFlow is a web app with:

- Browser client (React)
- Backend service (Node/Express)
- MongoDB database

The system targets major browsers (Chrome/Firefox) and is designed to be set up by developers without heavy infrastructure.

*Note: The UI is desktop-first; mobile/tablet responsiveness is not in the current scope.

1.3 Document Purpose & Audience

This document is a **developer's manual** (design + implementation guide), focused on architecture, component interactions, and key algorithms/flows.

It is written for developers who will maintain or extend TaskFlow.



1.4 Related Documents

- TaskFlow SRS (Group 02 - Requirements Specification)
- Testing document (Group 02 - Testing)
- GitHub repository: source code + commit history (required for auditability)
- Course project requirements (format + deliverables)

2. System Architecture

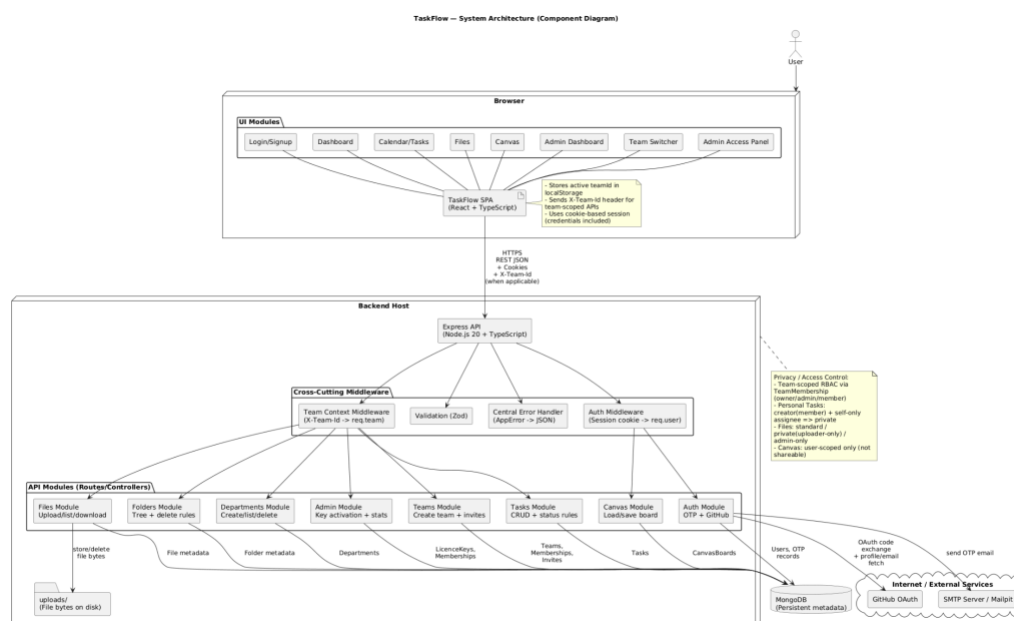
2.1 High-Level Architecture Overview

TaskFlow uses a standard **3-layer web architecture**:

1. **Client (Browser)**
 - React SPA providing login flow, navigation, and feature pages.
2. **Service Layer (Backend APIs)**
 - Express API server implementing authentication, authorization, team context, admin key licensing, task/file/canvas logic.
3. **Database Layer**
 - MongoDB storing user accounts, team membership/roles, tasks, files, metadata, folder structure, departments, license keys, and canvas boards.

*All access to persisted data occurs through the backend service.

UML Diagram for System Architecture





2.2 Technology Stack

Frontend:

- React + TypeScript
- Vite
- Tailwind CSS
- React Router
- Markdown preview: react-markdown + remark-gfm
- DOCX preview: Mammoth (client-side conversion)

Backend:

- Node.js (target runtime: Node 20)
- Express
- MongoDB via Mongoose
- Zod validation for request payloads
- JWT session cookie (httpOnly)
- Multer for uploads
- Nodemailer for OTP mail delivery (Mailpit supported for development)
- Security middleware: Helmet, CORS, rate limiting

Database:

- MongoDB local (development)
- Collections designed around team-scoped access and privacy constraints

CI / Tooling:

- Prettier formatting
- ESLint
- Typecheck (TypeScript)
- Backend tests: Node test runner (tsx)
- Frontend tests: Vitest
- E2E tests: Playwright
- GitHub Actions CI workflow

2.3 Deployment Model

Intended for:

- Local development (MongoDB local, optional Mailpit)
- Demonstration environment (same architecture; production hardening is out of scope)



3. Database Design

3.1 Database Selection & Rationale

MongoDB is used because TaskFlow primarily stores document-like entities (tasks, folders, files metadata, canvas boards) and benefits from flexible evolution as features grow.

3.2 Data Model Overview (Collections)

Below is the effective schema used in the implementation (high-level):

Users:

- `_id`
- `email`
- `role` (global coarse role: user/admin for session gating; team roles are stored separately)
- `githubId`, `githubUsername` (optional)
- `audit fields(createdAt/updatedAt)`

Teams:

- `_id`
- `name`
- `ownerUserId`
- `createdAt/updatedAt`

TeamMembership:

- `_id`
- `teamId`
- `userId`
- `role` (owner/admin/member)
- `constraints: one membership per (teamId, userId)`

LicenceKeys (Admin Keys):

Course handout describes license keys and licensing schemes as a required module.

TaskFlow implements team activation/admin access using a key format **AAAA-BBBB-CCCC** (4-4-4).

The course handout shows **AAAA-BBBB-CCCC-DDDD** as an example format; TaskFlow standardizes to 4-4-4 by agreement with course staff and for usability.

Key fields:

- `_id`
- `key` (string “AAAA-BBBB-CCCC”)



- teamId (nullable until the owner finishes team creation)
- ownerId (set on first activation)
- maxUses, usesCount
- expiresAt
- revoked (boolean)

Tasks:

- _id
- teamId
- name, description
- dueDate
- priority (Low/Medium/High)
- department
- status (Not Started / In Progress / Completed)
- assignees (array of identifiers)
- creator (creator identifier)
- editedAt (set only for non-status edits)
- key rule: **personal tasks** are self-created and self-assigned and are invisible to other users.

Folders:

- _id
- teamId
- name
- parentId (nullable for root)
- isPrivate + ownership metadata (for private visibility)
- Used to build folder tree UI

Files (metadata)

- _id
- teamId
- folderId
- originalName, mimeType, sizeBytes
- storedName (server-side filename)
- access: standard | private | admin (admin-only)
- ownership metadata (needed for private access enforcement)

Departments

- _id
- teamId
- name



- Created/deleted by owner/admin; shared across team.

CanvasBoards

- `_id`
- `userId` (strictly personal, not shareable)
- serialized board state (nodes/edges/shapes)
- autosave timestamps

3.3 Indexing & Performance Notes

- TeamMemberships: (teamId, userId) unique
- LicenceKeys: key unique; plus index on expiresAt
- Tasks: (teamId, dueDate) and (teamId, assignees)
- Files/Folders: (teamId, folderId) and (teamId, access)

4. User Interface Design

4.1 UI/UX Design Principles

- Minimal visual noise; clear navigation
- Consistent interaction patterns across modules (table/list, filters, modal details)
- “Safe defaults” (members create personal tasks by default; private-by-design canvas)
- Explicit access boundaries (admin-only UI only appears to authorized users)

Top bar:

- Current user identit
- Team selector dropdown (switch active team context)

Bottom-left sticky box:

- Admin Access Panel (enter key to activate admin/owner role for a team)

4.2 Key Screens & Workflows

Login / Signup

- OTP request → OTP verification → session established
- Alternative: GitHub OAuth redirect → callback → session established

Team Selector

- User can switch between teams without re-authentication.
- Switching team updates client-side active team context and changes backend request headers.



Dashboard

- Activity Feed:
 - Built from visible task and file events (created/edited/status changes/uploads)
 - Clicking an item opens Task details modal if user has access
- Tasks Due Today
 - List of tasks due today (based on dueDate)
 - Clicking opens Task Details modal

Calendar

- Calendar view (default), list view, completion view
- Create task modal with required fields
- Filters: priority + department
- Status tick button to mark completion

Files

- Folder tree (left) + file list/grid (right)
- Upload file up to 50MB
- File access:
 - Standard: visible to all team members
 - Private: visible to uploader only
 - Admin-only: visible to owner/admin only
- Preview in browser (PDF/MD/DOCX/etc.)
- Delete confirmation warns how many items will be deleted; deletion is permanent

Canvas

- Personal board with shapes (Sticky/Card/Rectangle/Diamond) and connectors (Line/Arrow)
- Snap-to-grid toggle
- Undo/redo shortcuts
- Autosave (board persists after refresh/logout)

Admin Dashboard

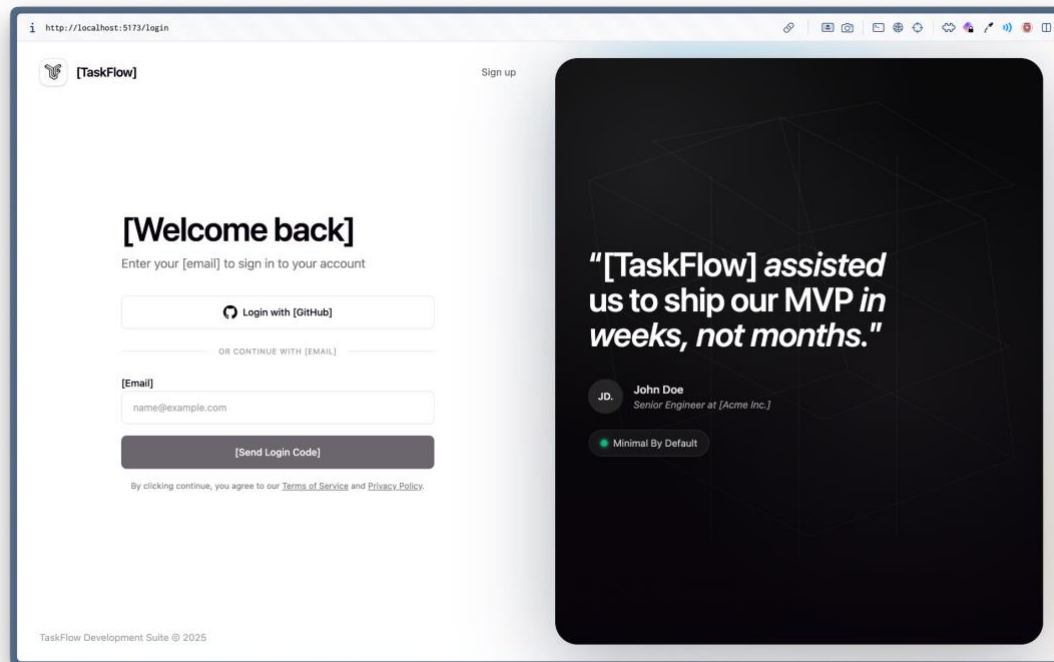
Visible only to owner/admin:

- Membership management
- Owner sees the admin key + remaining uses + expiry state
- Admin does not see key and cannot promote other admins

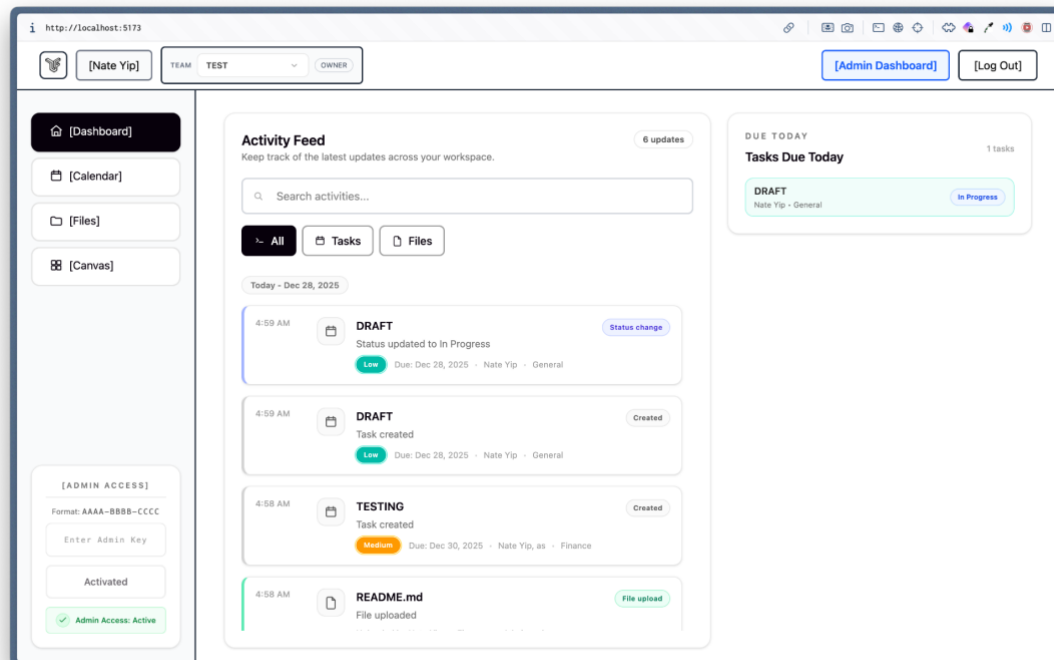


Wireframes

- Login page (OTP + GitHub option)

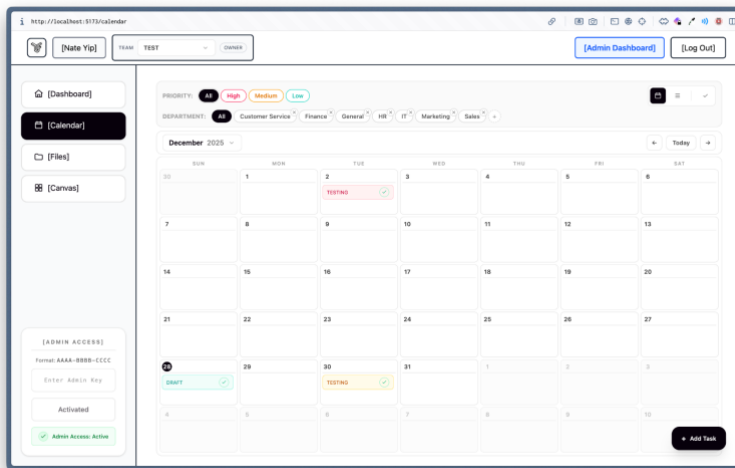


- Dashboard (feed + due today)

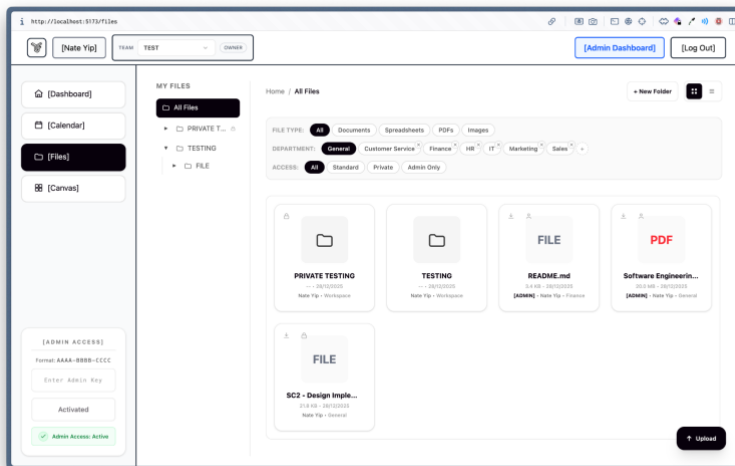




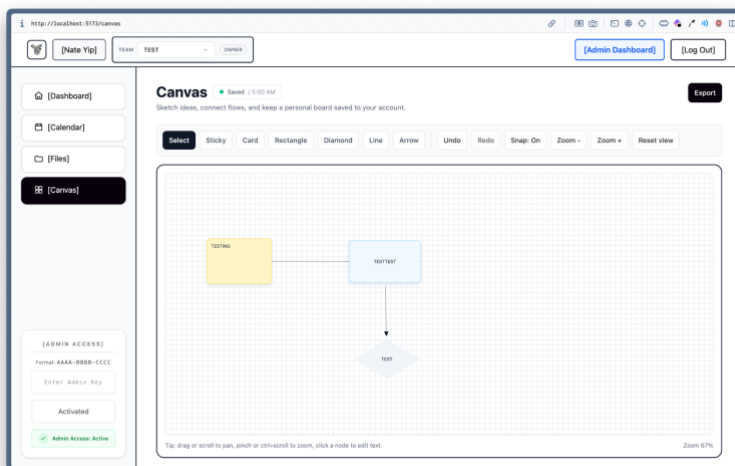
- Calendar (month + task details modal)



- Files (tree + grid/list + preview modal)



- Canvas board





5. Application-Specific Features

5.1 Feature 1 - Team-Scoped Roles with Admin Key Activation (Differentiator vs. “simple login-only” tools)

Purpose

Enforce controlled access to team administration and role assignment, using a license key mechanism (admin key). License management is a required module.

Behavior Summary

- Key format: **AAAA-BBBB-CCCC**.
- First activation of a fresh key:
 - user becomes **owner** (pending team creation)
 - user must name the team; until then, others attempting the same key are blocked
- Owner can reveal/share key and manage admins/members
 - rotation on expiry (new key is generated and linked to the team)

5.2 Feature 2 - Task Privacy Model (Personal vs. Team Tasks)

Purpose

Prevent accidental exposure of personal tasks while allowing admins/owners to manage team tasks.

Rules (as implemented)

- Normal member can create **personal tasks only** (self-created + self-assigned).
- Personal tasks are invisible to everyone else (including owner/admin).
- Admin/owner tasks can be assigned to team members.
- Status change is allowed for assignees even if they are not the creator.
- Only creator can edit details; non-status edits set “edited” indicator.

5.3 Feature 3 - File Management with Access Classes + In-Browser Preview

Purpose

Provide a team file area with clear access boundaries and strong UX.

Rules

- Max upload size: 50MB
- Access:
 - Standard: everyone in team
 - Private: uploader only
 - Admin-only: owner/admin only
- Folder privacy supported; admin-only folders are not used (only file-level admin restriction)
- Preview types:
 - PDF/image via browser rendering



- Markdown rendered in client
- DOCX converted client-side to HTML

5.4 Feature 3 - Personal Canvas (Primary Differentiator)

Purpose

Embed a lightweight Miro-like brainstorming tool inside TaskFlow without requiring external apps.

Rules

- Canvas is per-user; not shareable
- Snap-to-grid optional
- Keyboard shortcuts for undo/redo
- Autosave to persist across sessions

6. Implementation Details

6.1 Repository Structure

Monorepo layout (single GitHub repository required for auditing).

- /backend - Express API + MongoDB models
- /frontend - React SPA
- /e2e - Playwright tests + config
- /.github - CI workflows
- /docs - documentation and project artifacts

6.2 Backend Architecture (Express)

Layering

- **Routes:** define API surface per module
- **Middleware:**
 - Authentication (JWT cookie session)
 - Team context (active team from header)
 - RBAC/authorization guards
 - Rate limiting + security headers
- **Controllers:**
 - Auth (OTP, GitHub OAuth)
 - Teams (create team, invites/membership)
 - Admin (key activation, key stats, admin operations)
 - Tasks (CRUD, status updates, visibility)
 - Files/Folders/Departments (CRUD + access)
 - Canvas (load/save board states)
- **Models (Mongoose):** Users, Teams, Memberships, LicenceKeys, Tasks, Files, Folders, Departments, CanvasBoards



- **Utilities/Services:**
 - Key generation + formatting (4-4-4)
 - OTP sending/verification
 - GitHub OAuth integration

Session & Authentication

- OTP login
 - POST /auth/otp/request (email + purpose)
 - send code via SMTP (Mailpit-supported in dev)
 - POST /auth/otp/verify → sets session cookie
- GitHub OAuth login:
 - GET /auth/github/start → redirects to GitHub
 - callback validates state cookie
 - fetch verified email (or fallback logic)
 - find/create user record
 - set session cookie

Key Security Decisions

- httpOnly cookie sessions reduce XSS risk compared to localStorage tokens
- Rate limiting protects OTP endpoints from brute-force attempts
- Zod-based payload validation reduces malformed request risk
- Role/visibility checks are enforced server-side (never trust UI alone)

6.3 Admin Key (“Licence Key”) Lifecycle Algorithm

The admin key is a license management mechanism satisfying required licensing functionality.

Data invariants

- Each key has (maxUses, usesCount, expiresAt, revoked)
- Key may be “unbound” (teamId null) during the owner’s team creation window
- Owner visibility: only owner can view the raw key and remaining uses in Admin Dashboard
- Admin visibility: can manage members but cannot promote admins or view key material

Flow A — First Activation → Become Owner (Pending Team Creation)

1. User inputs key in Admin Access Panel.
2. Backend validates:
 - a. key exists and not revoked
 - b. usage capacity available
3. If key has no ownerUserId:
 - a. set ownerUserId = currentUser
 - b. increment usesCount
 - c. set current user coarse role to admin-capable



4. User completes “Create Team” by providing team name:
 - a. backend checks the key’s ownerId matches the requester
 - b. creates team and binds teamId to key
 - c. creates TeamMembership(role=owner)

Concurrency guarantee:

If another user tries to activate the same key while the owner hasn’t finished team creation, activation is blocked until team naming is completed (prevents ambiguous ownership assignment).

Flow B — Subsequent Activation → Become Admin of That Team

1. User activates key
2. Backend validates (same as above)
3. Because key has teamId and already has ownerId:
 - a. increment usesCount
 - b. upsert TeamMembership(role=admin) for that team
 - c. return success

Flow C — Expiry & Automatic Rotation

- Keys have TTL controlled by environment (ADMIN_KEY_TTL_DAYS)
- On admin dashboard access / stats retrieval, backend:
 - if expired, revokes old key and generates a fresh key
 - binds the new key to the same team, preserving usage policy intent

Flow D — Team Deletion

- Only owner can delete team.
- When a team is deleted:
 - delete team entity
 - delete or revoke the linked key record
 - membership records and team-scoped data are removed
- Deletion is irreversible (matches product behavior for files/folders as well).

6.5 Task Visibility & Edit Rules (Implemented Enforcement)

The task system is designed to prevent leakage of personal tasks while enabling team coordination.

Definitions

- Personal task: created by a normal member and self-assigned; visible only to creator.
- Team task: created by owner/admin and assigned to one or more users (or to someone other than creator).

Authorization Matrix (effective)

- Member:



- can create personal tasks only
- can view only tasks where they are assignee OR creator (personal)
- can change status if assignee
- cannot edit details unless creator
- Admin/Owner:
 - can view team tasks
 - cannot view other users' personal tasks
 - can edit team tasks (subject to creator/assignee rules defined by product logic)
 - can change status if assignee; can also manage tasks as admin where allowed

6.6 File Access Control & Preview Implementation

Storage design

- Actual file bytes stored on server disk
- Mongo stores metadata, access class, ownership, and folder placement

Access classes

- Standard: visible to team
- Private: visible only to uploader
- Admin-only: visible only to admin/owner

Preview pipeline (client-side)

- User clicks “Preview”
- Frontend fetches file bytes as blob
- Based on MIME/type:
 - PDF/images: generate object URL and render
 - Markdown: fetch text and render via markdown renderer
 - DOCX: convert to HTML client-side and render in modal
- Download uses direct file endpoint returning bytes with correct headers

6.7 Canvas Autosave

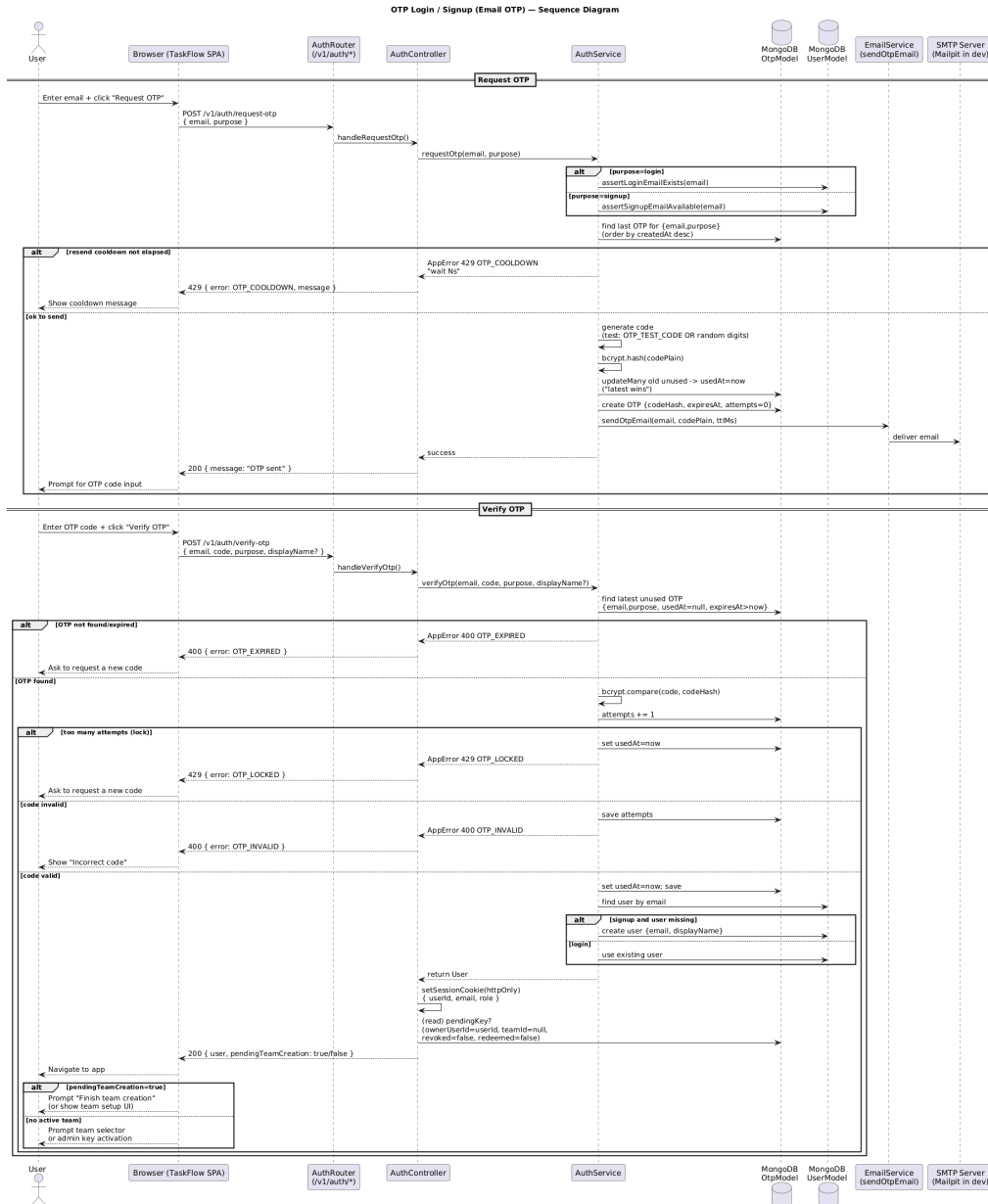
Canvas is per-user (not team-shared). It persists via:

- periodic autosave calls (or save-on-change)
- backend stores serialized board state in CanvasBoards collection keyed by userId
- on load, frontend requests last saved state and rehydrates UI



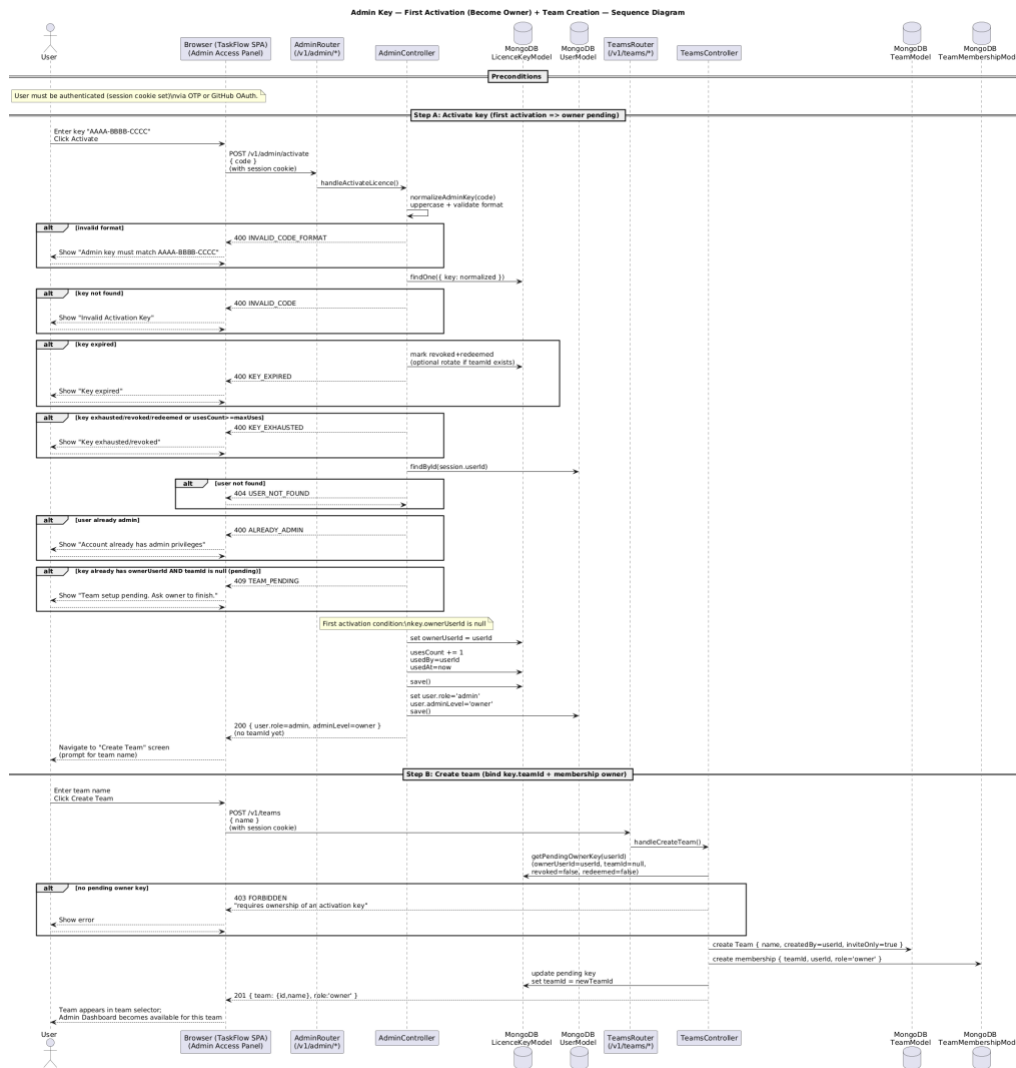
7. Dataflow & Processes

7.1 OTP Login Sequence (GitHub OAuth is similar)



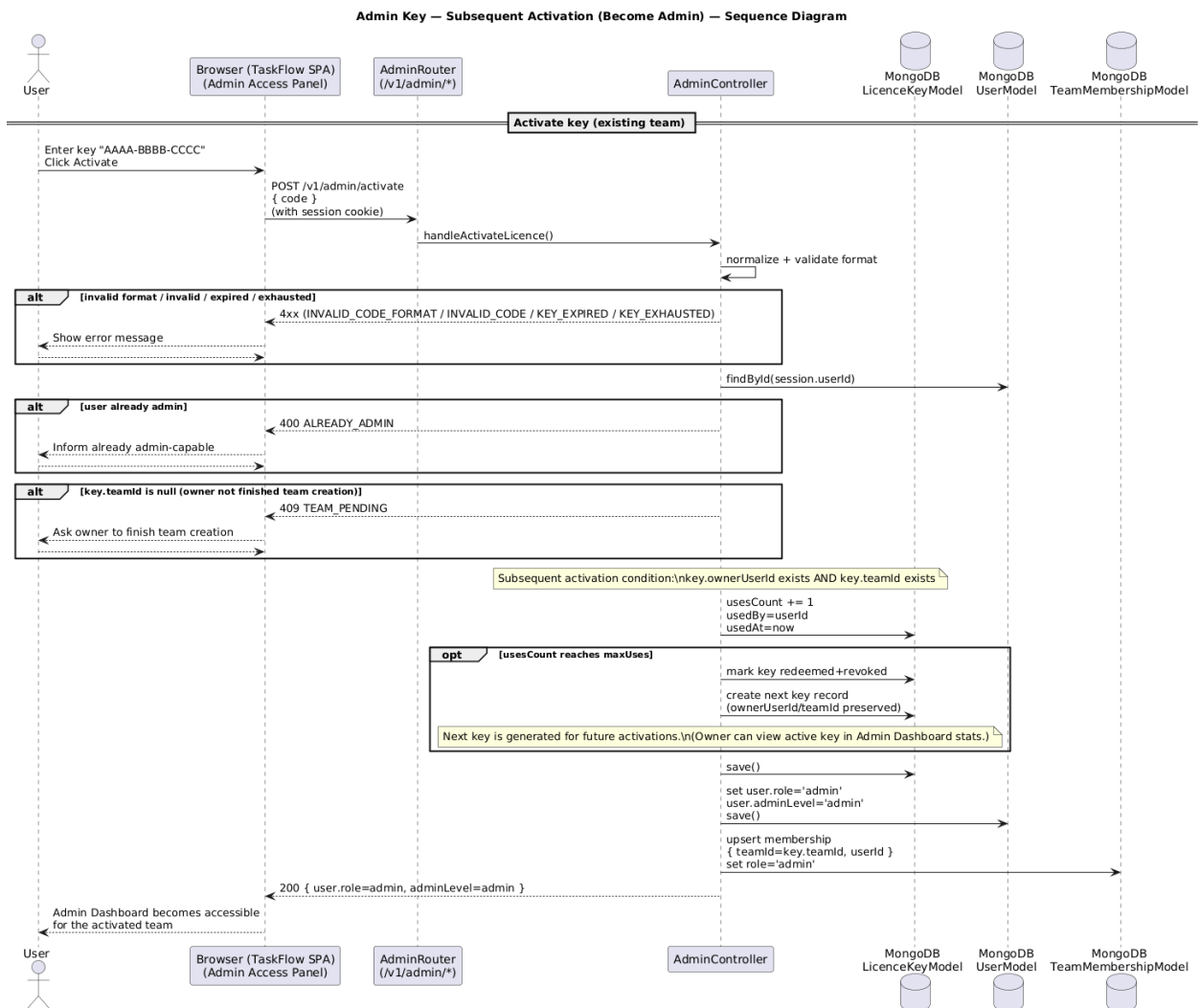


7.2 Admin Key First Activation + Team Creation



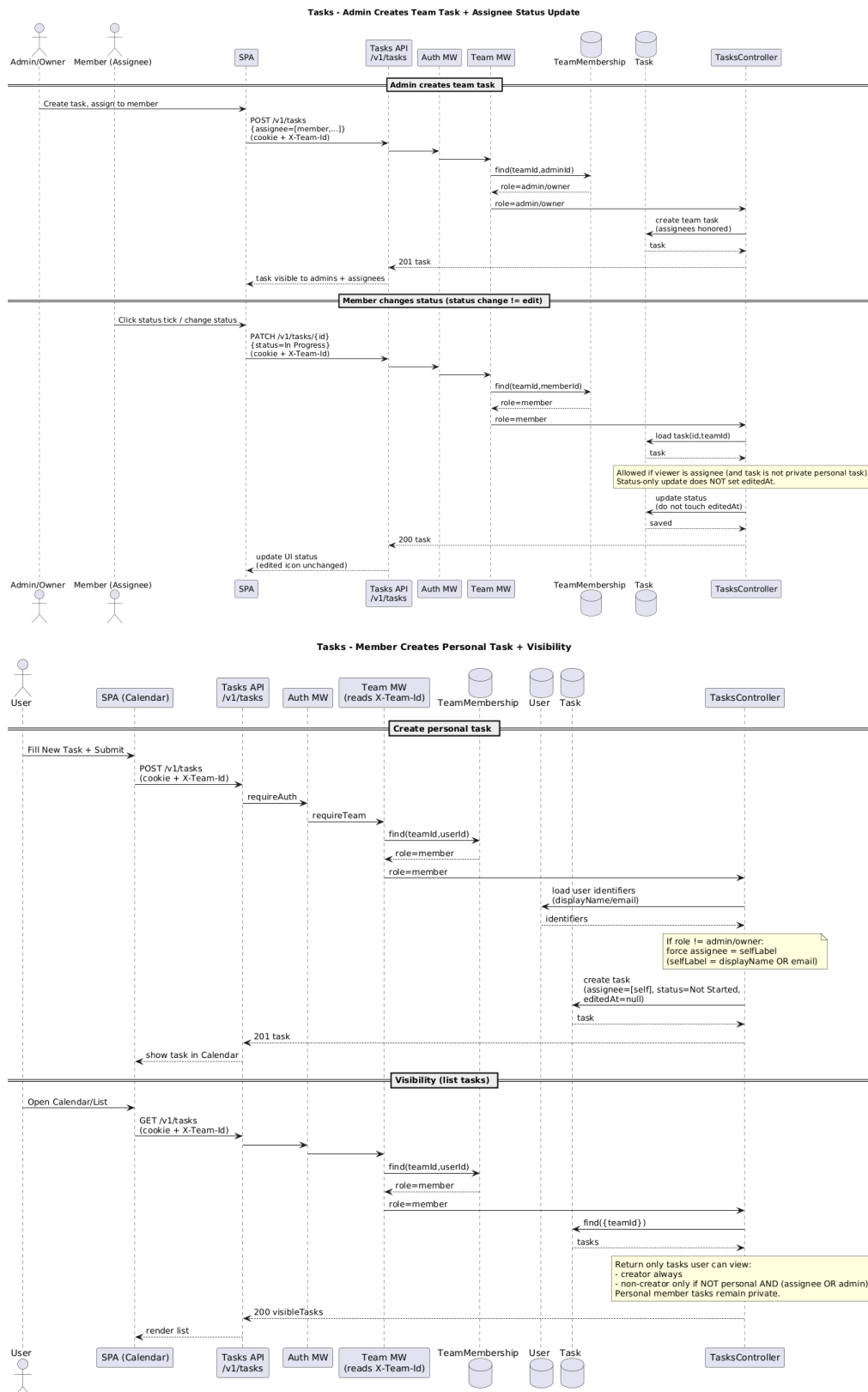


7.3 Admin Key Subsequent Activation



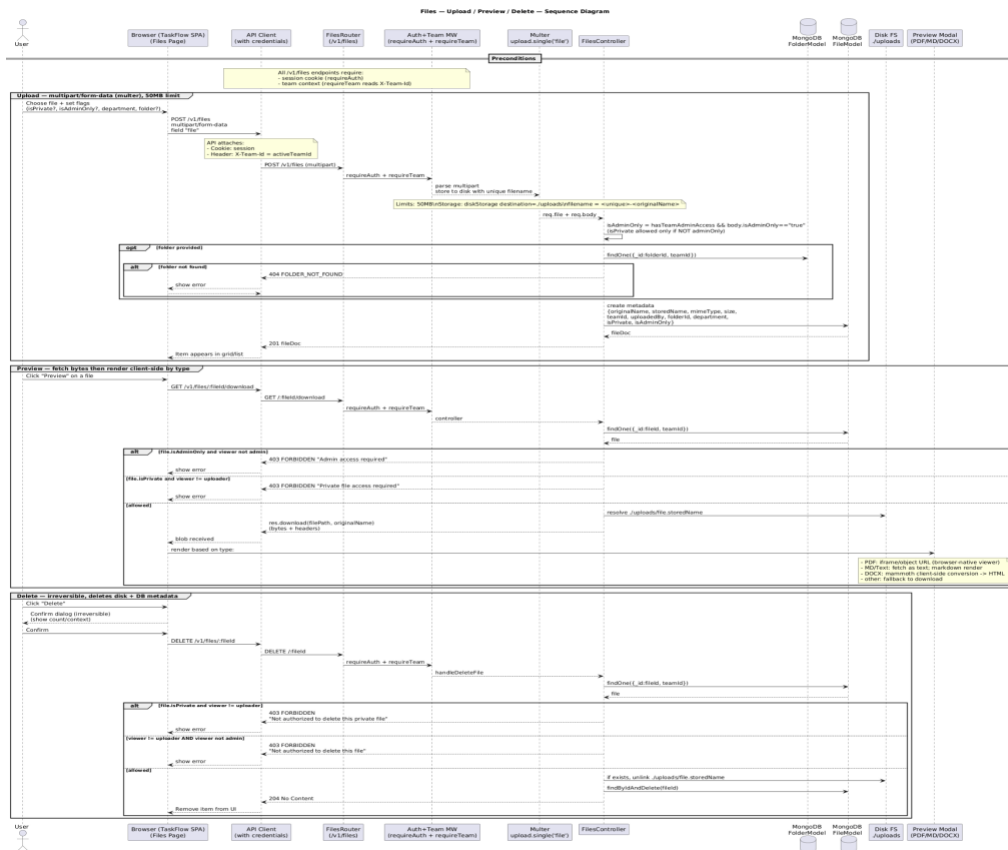


7.4 Task Create / Edit / Status Update

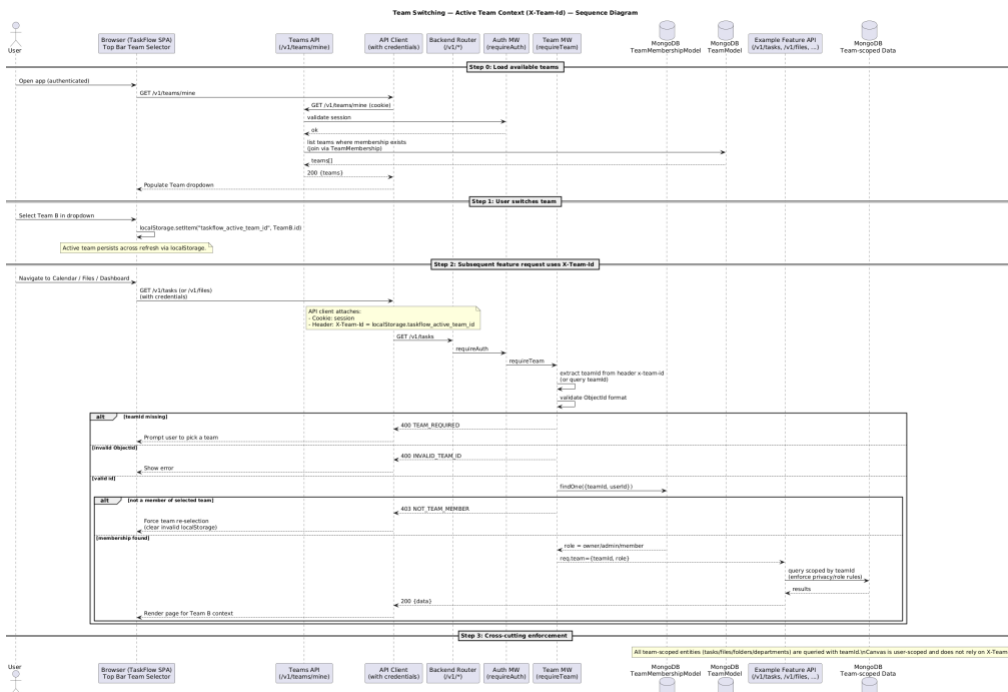




7.5 Files Upload / Preview / Delete



7.6 Team Switching





8. Developer Setup & Operations

8.1 Prerequisites

- Node.js v20
- MongoDB local instance
- Docker (optional; for Mailpit convenience)

8.2 Environment Variables

Backend “.env” supports:

- ADMIN_KEY_AUTO_SEED=true/false (developer convenience)
- INITIAL_ADMIN_KEY=ADMINKEY-2025 (developer convenience)
- ADMIN_KEY_TTL_DAYS=30
- ADMIN_KEY_MAX_USES=5
- GitHub OAuth config (client id/secret) if enabling GitHub login
- SMTP config (Mailpit in dev)

8.3 Common Commands

- Install all:
 - npm install
 - npm - prefix backend install
 - npm - prefix frontend install
- Run dev (both):
 - npm run dev
- Backend tests:
 - npm run test:backend
- Frontend tests:
 - npm run test:frontend
- E2E:
 - npm run playwright:install
 - npm run test:e2e



- Seed keys (developer tool):
 - node backend/scripts/seedLicence.mjs DEMO-KEYS-2025
 - Generate key (developer tool):
- npm -prefix backend run admin:key:generate

8.4 Running Mailpit (optional)

- docker compose -f docker-compose.mailpit.yml up -d
- npm run mail:up
- npm run mail:down

9. Acknowledgement

The authors employed ChatGPT 5.2 to assist in the refining prose, enhancing clarity, and ensuring consistent formatting throughout this document. The core ideas, and structure originate from the authors' knowledge, and research.

Appendix. GitHub Link & Revision History

GitHub Repo: <https://github.com/lukecarroll15/CSCI3100-Project>

Current version: v1.2

Document status: Final

Version	Date (DD/MM/YYYY)	Author(s)	Description of Changes
v0.8	20/11/2025	Archie Hamilton; Luke Carroll	Initial draft completed (design and implementation baseline established).
v1.0	26/12/2025	Chun Wang YIP	Finalized content revision; design sections completed and aligned with implemented system.
v1.1	27/12/2025	Archie Hamilton	Formatting and layout pass to ensure document consistency and submission readiness.
v1.2	28/12/2025	Chun Wang YIP	Final revision and formatting; final quality checks and polishing for submission.